

第一阶段工作小结

前言

本阶段的核心目标，不是尽快把“登录注册”“旅游规划”“历史记录”等业务功能堆起来，而是先把项目的基础工程能力搭稳。对于一个课程全栈项目来说，最容易犯的错误就是一开始把注意力全部放在功能页面和接口数量上，结果到真正联调时才发现前后端结构混乱、环境配置不统一、数据库接入方式不稳定、开发脚本不规范，最后大量时间被浪费在返工和救火上。

因此，这一阶段我刻意把工作边界控制在“项目骨架与开发环境搭建”之内，聚焦完成以下几件真正影响后续开发效率的事情：前后端独立工程初始化、基础依赖选型与安装、Prisma + SQLite 接入、后端健康检查接口实现、前端最小联通性验证页面搭建，以及必要的 README 和运行说明补充。换句话说，这一阶段做的是“打地基”，不是“盖房子”。

从结果来看，这一阶段已经达成了预期目标：项目具备了前端、后端、数据库三者之间的最小闭环，后续可以在这个稳定骨架上继续推进业务模型设计、注册登录、页面路由和旅游规划能力开发，而不需要再回头重做底层结构。

1. 本阶段工作内容（包括环境配置）

1.1 项目目录结构与工程组织方式

本阶段首先明确了项目采用前后端分离的目录结构，并将仓库划分为两个独立子项目：

- frontend：前端工程目录
- backend：后端工程目录

之所以采用这种“双目录、独立管理”的方式，而不是一开始就上 monorepo、workspace、shared 包或者更多公共基础设施，主要基于两个考虑。

第一，当前阶段的任务是尽快形成一个最小可运行版本。对课程项目来说，前期最重要的是把技术链路打通，而不是为了“架构看起来高级”就引入额外复杂度。前后端分别维护自己的 package.json、脚本和依赖，启动逻辑清晰，出问题时定位范围也更小。

第二，这种组织方式很适合单人或小团队迭代。后续如果项目规模扩大，确实可以再考虑根目录 workspace、共享类型包或者统一脚本，但在第一阶段提前做这些，收益远低于成本。工程设计要服务当前阶段目标，而不是为了未来所有可能性做过度预埋。

1.2 前端环境初始化

前端部分使用了以下技术栈：

- 开发语言：TypeScript
- 前端框架：React
- 构建工具：Vite
- 样式工具：Tailwind CSS

初始化完成后，前端具备了标准的开发和构建能力，保留了最基础的脚本约定：

- `npm run dev`：启动本地开发服务
- `npm run build`：执行 TypeScript 编译并完成生产构建
- `npm run preview`：预览构建产物

在页面层面，本阶段没有开发任何业务页面，而是只保留了一个极简状态页。这个页面的目的非常明确：验证前端是否能成功请求后端接口，并根据返回结果判断数据库是否已经打通。换句话说，它不是产品页面，而是工程联调页面。

这样做的价值在于，前端在第一阶段不必承受过多的页面设计压力，也不需要为了“看起来像完整系统”而提前写一堆未来可能推翻的布局代码。先把请求链路打通，比先画一堆静态页面更有价值。

1.3 后端环境初始化

后端部分采用了以下技术栈：

- 运行环境：Node.js
- 后端框架：Express
- 开发语言：TypeScript

后端保持了“最小但可扩展”的结构，主要分为以下几个部分：

- `src/app.ts`：负责创建 Express 应用并挂载中间件和路由
- `src/server.ts`：负责启动服务
- `src/routes/health.ts`：负责健康检查接口
- `src/lib/prisma.ts`：负责 PrismaClient 初始化与数据库连接封装

同时也补上了最基本的脚本能力：

- `npm run dev`：开发模式运行后端
- `npm run build`：编译 TypeScript 代码
- `npm run start`：运行编译后的产物

- `npm run prisma:generate` : 生成 Prisma Client
- `npm run prisma:migrate:dev` : 后续本地迁移开发使用
- `npm run prisma:migrate:deploy` : 后续部署环境迁移使用

这一阶段后端的重点不在于接口数量，而在于把未来可持续扩展的骨架搭出来。Express 本身很轻，适合课程项目起步；而 app 与 server 分离、数据库连接单独抽离、路由独立管理，这些看起来只是“小规范”，但对后续测试、调试、拆模块、补中间件都会有很大帮助。

1.4 CORS 与开发联调策略

后端在本阶段增加了最基础的 cors 配置，并通过白名单允许本地前端开发地址访问，主要包括：

- `http://localhost:5173`
- `http://127.0.0.1:5173`

这里的设计并不是让前端开发阶段完全依赖 CORS，而是采用“双保险”策略：

- 前端开发默认优先通过 Vite Proxy 访问后端
- 后端保留最小 CORS 白名单，作为本地调试和直接访问接口时的兜底

这种处理方式的好处是很实际的。前端用 Proxy 能保持调用路径统一，例如统一请求 `/api/health`，减少环境分支判断；而后端补一个最小 CORS 配置，则可以避免调试阶段因为跨域问题浪费时间。工程实践里，类似这种“简单但可靠”的兜底设计，往往比过于理想化的单方案更稳。

1.5 Prisma 与 SQLite 的接入

数据库部分本阶段使用：

- 数据库：SQLite
- ORM：Prisma

这一阶段虽然没有正式设计业务表，但已经完成了 Prisma 的基础接入，包括：

- 配置 `prisma/schema.prisma`
- 配置 `prisma.config.ts`
- 配置 `.env` 和 `.env.example`
- 执行 `prisma generate`
- 生成本地 SQLite 数据库文件

这里的重点不是“数据库里存了什么”，而是“数据库通路已经建立起来了”。对于第一阶段来说，只要能确保后端可以稳定地通过 Prisma 操作 SQLite，并且未来能够按标准 migration 流程继续建模，这一步的目标就已经达成。

为了兼顾当前需求和后续规范，本阶段还明确了 Prisma 的标准工作流：

1. 当前阶段先完成配置和 `prisma generate`
2. 下一阶段开始正式建模时，在 `schema.prisma` 中加入业务模型
3. 使用 `prisma migrate dev --name <name>` 生成并应用迁移
4. 后续部署环境使用 `prisma migrate deploy`

这一点很重要。课程项目里很常见的一种情况是：前期为了省事直接改库，后期字段一多就无法追踪结构演化过程。把 migration 流程从第一阶段就讲清楚，后面会省掉很多不必要的结构混乱。

1.6 健康检查接口设计与实现

后端本阶段新增了一个非常关键的接口：

```
GET /api/health
```

这个接口的意义远不只是“能返回 200”。它承担了三层验证职责：

1. 验证后端服务本身能否启动
2. 验证路由层是否工作正常
3. 验证数据库连接是否真实可用

接口返回内容采用了尽量简洁但足够有用的结构，包括：

- `status`
- `service`
- `database`
- `timestamp`

并且，这个接口不是返回静态字符串，而是通过 Prisma 执行了最小数据库探测（如 `SELECT 1`），在成功时返回数据库可用状态，在失败时返回错误信息。这个设计有两个现实意义：

第一，它让前端看到的“系统可用”是真实的，而不是假象。如果只是简单返回 `{ status: "ok" }`，那么后端即便没有连上数据库，前端也会误判系统已经可用。

第二，它为后续监控和排障留下了良好基础。哪怕只是课程项目，一个能区分“服务挂了”和“数据库挂了”的健康检查，也比纯静态接口更有价值。

1.7 前端首页极简状态页

前端首页本阶段只实现了一个极简状态展示页。页面逻辑分为三种状态：

- loading：正在检查后端和数据库连接
- success：显示后端和数据库状态正常
- error：显示失败提示

这个页面的设计原则非常克制：

- 不追求完整产品界面
- 不引入复杂组件库
- 不展示大段原始 JSON
- 只保留对当前阶段最有价值的信息

这样做的好处是，第一阶段的前端页面职责非常单一，任何人打开页面都能立刻看懂系统现在处于什么状态。工程上，联调页面最怕“做得很华丽，但看不出到底哪里通了、哪里没通”。极简状态页虽然朴素，但信息密度更高，也更适合作为骨架阶段的验证手段。

1.8 README 与基础说明补充

除了代码本身，本阶段还补充和整理了项目说明，包括：

- 技术栈说明
- 当前阶段完成情况
- 前后端启动方式
- /api/health 接口说明
- Prisma 的 generate / migrate dev / migrate deploy workflow 说明
- 下一阶段建议

很多课程项目到最后才补 README，结果往往是“代码能跑，但别人不知道怎么跑”。实际上，文档和代码一样，都是工程的一部分。哪怕只是一个最小骨架阶段，只要已经明确了启动脚本、技术栈和后续迁移流程，就应该及时写下来。这样后面不管是自己回顾，还是给老师、同学展示，成本都会低很多。

1.9 本阶段最终成果

综合来看，本阶段已经完成了以下关键成果：

- 建立了前后端分离的双目录项目结构
- 初始化了前端与后端工程
- 确立了 React + TypeScript + Vite + Tailwind CSS 的前端基础栈
- 确立了 Node.js + Express + TypeScript 的后端基础栈
- 接入了 Prisma + SQLite
- 完成了 Prisma Client 生成和本地数据库接入
- 增加了最小 CORS 配置

- 实现了真实验证数据库可用性的 `GET /api/health`
- 完成了前端极简状态页，用于验证联通性
- 补充了 README 与基础运行说明

这意味着项目已经从“只有一个想法和目录”迈入“具备可运行骨架和可持续迭代基础”的状态。这个阶段没有业务亮点，但它决定了后续开发能否顺利推进。

2. 遇到的问题以及解决思路

这一阶段虽然功能不多，但工程问题其实并不少。很多问题看起来只是配置细节，实际上会直接影响后续开发效率和系统稳定性。下面挑几个最有代表性的点来总结。

2.1 如何保持最小依赖，避免第一阶段变成“配置工程”

2.1.1 现象

项目刚开始搭建时，很容易出现一种倾向：既然要做全栈项目，不如把路由、状态管理、UI 库、鉴权框架、请求封装、日志系统、校验框架、测试框架、Docker、monorepo 等全部一次性铺好。

2.1.2 原因分析

这种做法表面上看是“提前规划”，实际上往往会把第一阶段变成无休止的环境配置工作。更严重的是，很多配置在项目早期并没有被真实需求验证，等后面业务细节清晰后，前面做的大量预埋反而会变成包袱。

2.1.3 处理思路

本阶段采取了非常克制的依赖策略：

- 前端只保留 React、Vite、TypeScript、Tailwind 所需最小依赖
- 后端只保留 Express、TypeScript、Prisma、SQLite 相关依赖
- 不提前引入状态管理库、路由体系、组件库和复杂中间件
- 不上 monorepo、共享包或统一根脚本

2.1.4 最终结果

项目在保持结构清晰的同时，没有被大量无关配置淹没。第一阶段的重点始终放在“跑起来”和“打通链路”上，这样做虽然不花哨，但非常适合课程项目的实际节奏。经验上，稳定的最小版本，永远比半成品的“未来大架构”更有价值。

2.2 后端 TypeScript 构建配置与新版本兼容问题

2.2.1 现象

在后端构建过程中，TypeScript 的模块解析配置与较新版本工具链之间出现了兼容问题，导致构建阶段报错。

2.2.2 原因分析

这类问题本质上是工具链升级后的默认行为变化造成的。很多旧配置在以前能跑，并不代表在新版本 TypeScript、Node 或构建工具组合下仍然是最稳妥的写法。骨架阶段如果忽略这种问题，后面功能一多，构建错误会越来越难定位。

2.2.3 处理思路

这一阶段没有采取“临时忽略”或者“强行压版本”的方式，而是直接把后端 `tsconfig` 调整到更符合当前 Node 运行方式的模式，确保构建脚本本身是干净可用的。

2.2.4 最终结果

后端构建链路恢复稳定，`npm run build` 能正常执行。这类问题的处理原则很简单：既然问题已经暴露，就在骨架阶段解决，不要把技术债拖到业务开发期。越往后拖，修复成本越高。

2.3 Prisma 7 + SQLite 在运行期需要 driver adapter

2.3.1 现象

一开始以为完成 `@prisma/client` 安装、`schema.prisma` 配置和 `prisma generate` 之后，后端就可以正常连接 SQLite，但实际运行时出现了 Prisma Client 初始化相关问题。

2.3.2 原因分析

问题的关键在于，较新的 Prisma 版本在某些运行模式下，仅有 `@prisma/client` 已经不足以支撑 SQLite 的运行期连接，还需要显式引入对应的 driver adapter。这个问题很容易被忽略，因为“生成成功”并不等于“运行成功”。

2.3.3 处理思路

本阶段补充了官方的 SQLite adapter，并将 Prisma Client 的初始化方式调整为显式使用该 adapter。这样做的意义不只是“解决当前报错”，更重要的是把 Prisma 的运行逻辑说明白了：生成 client 是一回事，运行期驱动是否正确接上是另一回事。

2.3.4 最终结果

后端能够通过 Prisma 对 SQLite 发起真实查询，健康检查接口也能基于数据库探测返回正确状态。这个问题说明一个很典型的工程经验：接入 ORM 时，不能只看安装和代码提示是否正常，必须至少跑通一条真实查询链路。

2.4 Prisma schema 配置与运行时 SQLite 路径的一致性问题的

2.4.1 现象

即使 Prisma 已经生成成功，如果 schema 配置中的数据库地址和运行时代码实际解析出的 SQLite 文件路径不一致，仍然可能出现“配置看上去没问题，运行起来却找不到库或连错库”的情况。

2.4.2 原因分析

SQLite 是文件型数据库，它不像远程数据库那样靠主机和端口区分实例。只要路径处理不一致，就可能导致：

- 开发时连的是一个库
- 运行时用的是另一个库
- 甚至压根没有连到预期文件

这一步之所以容易踩坑，是因为“看起来都是 `file:./dev.db`”，但不同运行上下文下的相对路径基准可能不同。

2.4.3 处理思路

本阶段对 Prisma 运行时连接进行了显式处理，确保运行时最终解析出的 SQLite 路径和 schema 配置所指向的数据库文件保持一致。同时，也在 README 中把这套工作流讲清楚，避免后续迭代时再出现“改了配置但连的不是同一个库”的问题。

2.4.4 最终结果

数据库路径的行为变得可预期，健康检查与 Prisma 工作流也更稳定。这个问题的经验教训是：凡是涉及“相对路径 + 运行时解析”的地方，都不应该想当然，最好在第一阶段就把行为固定下来。

2.5 健康检查不能只返回静态字符串

2.5.1 现象

如果只是为了“让前端有东西可请求”，后端完全可以写一个非常简单的接口，返回 `{ status: "ok" }`。但这种接口虽然能通过表面联调，却无法证明数据库真的可用。

2.5.2 原因分析

静态健康接口的问题在于，它只能证明 Web 服务进程起来了，不能证明依赖链路是通的。对于一个后续一定会依赖数据库的全栈项目来说，这种“伪联通”其实没有太大工程价值。

2.5.3 处理思路

本阶段把健康检查设计为真实数据库探测接口。也就是说，只有当后端服务可用、Prisma Client 正常、SQLite 连接成功时，接口才返回完整的健康状态；否则直接返回错误。

2.5.4 最终结果

前端状态页看到的不是“假正常”，而是对当前系统最关键依赖的真实反馈。这种设计虽然比静态接口多了一点工作量，但非常值得，因为它能让后续每一步开发都建立在更真实的基础之上。

3. 下一阶段计划和开发方案

第一阶段已经完成了骨架和联通性验证，接下来就不能继续停留在“环境搭建”层面，而需要进入真正有业务价值的设计和开发阶段。但下一阶段也不能盲目展开，仍然需要遵循“先打主链路、再补附加能力”的原则。

3.1 第一优先级：设计核心数据模型并完成首个正式 migration

下一阶段最先要做的，不是页面，而是数据模型。因为后续的注册登录、偏好卡片、历史记录和方案优化，本质上都离不开稳定的数据结构。

建议优先落地以下 3 个核心模型：

- User
- PreferenceCard
- TravelRecord

其中：

- User 负责承载账号信息和基础身份能力
- PreferenceCard 负责承载出游风格、交通方式、景点偏好、出发城市、天数等长期可复用配置
- TravelRecord 负责承载生成结果、历史记录和后续优化的基础数据

这一步完成后，应正式走 Prisma migration 流程，而不是手动改库。开发顺序建议如下：

1. 在 `schema.prisma` 中补齐模型定义
2. 执行 `prisma migrate dev --name init`

3. 检查数据库表结构和字段含义是否合理
4. 再开始进入接口开发

这一步做好了，后面的接口开发会轻松很多；如果模型没想清楚就先写接口，后面几乎一定会频繁返工。

3.2 第二优先级：实现注册/登录接口骨架

数据模型明确之后，下一步应优先实现最基础的注册登录能力。原因很简单：后续的偏好卡片、历史记录、方案优化，本质上都依赖“用户身份”这一前提。

这一阶段的目标不是做完整账号体系，而是做一个最小可用版本，包括：

- 注册接口
- 登录接口
- 密码摘要存储
- 基础用户数据读写
- 统一的接口返回结构

如果节奏允许，还可以顺手把鉴权方案先定下来，例如基于 session 或 JWT 做一个最小实现，但不建议一开始把权限系统做得太复杂。对课程项目来说，越早把“身份体系”搭起来，后续功能就越容易围绕用户维度展开。

3.3 第三优先级：建立前端基础路由与公共布局

在后端基础数据和身份体系开始成形后，前端就不应该继续停留在单页状态展示模式，而应该开始进入真正的页面组织阶段。

建议下一阶段前端重点做以下几件事：

- 建立页面路由结构
- 预留首页、登录页、偏好卡片页、历史记录页等入口
- 设计一个简单但统一的公共布局
- 抽离基础请求封装和错误处理逻辑

这里仍然建议保持克制，不要急着把所有页面一次性做满。更合理的做法是先把骨架路由和通用布局搭出来，然后随着接口逐步补充页面功能。页面结构先稳定，后续业务填充才会顺畅。

3.4 开发原则与推进方式

结合第一阶段的经验，下一阶段建议继续坚持以下开发原则：

3.4.1 继续坚持最小可运行迭代

每一轮开发都尽量形成一个可以运行、可以验证、可以演示的小闭环，而不是一次性铺开过多功能。比如“用户注册 + 登录 + 登录态验证”本身就可以作为一轮完整交付。

3.4.2 先打通主链路，再做附加能力

对于这个项目来说，主链路可以理解为：

用户注册/登录 -> 管理偏好卡片 -> 发起规划请求 -> 保存历史记录 -> 查看结果

天气信息、分享图、复杂优化策略等能力都很重要，但它们属于增强项，不应先于主链路开发。

3.4.3 数据模型、接口设计、页面结构同步推进

工程实践里最怕三件事脱节：

- 数据模型已经改了，接口还没跟上
- 接口返回变了，前端页面还按旧结构写
- 页面已经做复杂了，底层数据却不稳定

因此建议下一阶段以“模型 -> 接口 -> 页面”联动推进，每完成一层就做一次小闭环验证，而不是各自独立往前冲。

3.5 下一阶段的预期交付结果

如果按上面的顺序推进，下一阶段比较合理的交付结果应该包括：

- 首批 Prisma 业务模型落地
- 首个正式数据库 migration 完成
- 注册/登录接口可用
- 用户密码采用摘要存储
- 前端具备基础路由和公共布局
- 至少有一个真正面向业务的页面可以开始承接后续功能

这会让项目从“工程骨架可用”进入“业务主链路开始成形”的阶段，也是真正从搭环境过渡到做产品能力的关键一步。

结语

回头看第一阶段，工作量虽然主要集中在初始化、配置和联通性验证上，但这些内容并不“基础到不重要”，恰恰相反，它们决定了后面开发是顺着主线稳定推进，还是不断被环境、构建、数据库和接口问

题拖住。

一个项目能不能做顺，往往不是看第一周写了多少业务功能，而是看第一周有没有把工程地基打稳。本阶段的价值，就在于把前端、后端、数据库、运行说明和开发流程这些最底层的环节先理顺了。后续只要继续保持“最小可运行、逐步迭代、主链路优先”的节奏，项目开发会越来越顺，返工成本也会明显降低。